

AWS SECURITY

ULTRA-DEEP DIVE

A Comprehensive Reference for Cloud Security Professionals

Covering: Fundamentals | IAM | Network Security | Data Protection
Compliance | Monitoring | Incident Response | Real-World Case Studies

Prepared by: DevOps Shack | @devopsshack
Edition: 2026 | Version 1.0

TABLE OF CONTENTS

1 AWS Security Fundamentals	4
1.1 The AWS Shared Responsibility Model	4
1.2 AWS Global Infrastructure & Security	5
1.3 Core Security Principles in AWS	6
1.4 The AWS Well-Architected Security Pillar	7
2 Identity and Access Management (IAM)	9
2.1 IAM Fundamentals: Users, Groups, Roles, Policies	9
2.2 IAM Best Practices	10
2.3 AWS Single Sign-On & Identity Center	11
2.4 Multi-Factor Authentication (MFA)	12
2.5 Service Control Policies & Permission Boundaries	13
3 Network Security	14
3.1 Amazon VPC Architecture & Security	14
3.2 Security Groups and Network ACLs	15
3.3 AWS WAF, Shield, and DDoS Protection	16
3.4 AWS Network Firewall & Traffic Inspection	17
3.5 VPN, Direct Connect, and Private Connectivity	18
4 Data Protection	19
4.1 Encryption at Rest	19
4.2 Encryption in Transit	20
4.3 AWS Key Management Service (KMS)	21
4.4 AWS Secrets Manager & Parameter Store	22
4.5 Data Classification and S3 Security	23
5 Security Best Practices for Core AWS Services	25
5.1 EC2 Security Hardening	25
5.2 RDS and Database Security	26
5.3 Lambda and Serverless Security	27
5.4 Container Security (ECS/EKS)	28
5.5 S3 Bucket Security Best Practices	29
6 Compliance and Regulatory Standards	31
6.1 AWS Compliance Programs Overview	31
6.2 PCI DSS on AWS	32
6.3 HIPAA on AWS	33
6.4 SOC 2 on AWS	34
6.5 GDPR and Data Residency on AWS	35
7 Security Monitoring and Auditing	36
7.1 AWS CloudTrail	36
7.2 Amazon GuardDuty	37
7.3 AWS Security Hub	38
7.4 Amazon Macie	39
7.5 AWS Config and Compliance Monitoring	40

8 Incident Response on AWS	41
8.1 AWS Incident Response Framework	41
8.2 Forensics and Investigation Tools	42
8.3 Automated Remediation with Lambda & EventBridge	43
9 Case Studies	44
9.1 Capital One Data Breach Analysis	44
9.2 Securing a Multi-Account Enterprise on AWS	45
9.3 Building a Zero-Trust Architecture on AWS	46
10 Tools, Resources & Next Steps	47
10.1 AWS Native Security Toolbox	47
10.2 Third-Party Security Tools	48
10.3 Recommended Learning Path	49

01**AWS Security Fundamentals**

Amazon Web Services (AWS) is the world's most broadly adopted cloud platform, offering over 200 fully-featured services from data centers globally. As organizations migrate workloads to the cloud, security becomes a shared and critical responsibility. This section lays the groundwork for understanding how AWS approaches security at every layer.

1.1 The AWS Shared Responsibility Model

The most fundamental concept in AWS security is the Shared Responsibility Model. It divides security duties between AWS and the customer — AWS secures the cloud infrastructure itself, while customers are responsible for what they put inside it.

KEY CONCEPT: Shared Responsibility

AWS is responsible for security **OF** the cloud (hardware, software, networking, and facilities). Customers are responsible for security **IN** the cloud (data, applications, operating systems, network configuration, and identity management).

What AWS Manages

- Physical data center security (guards, cameras, biometrics, multi-factor access)
- Hardware: servers, storage devices, networking equipment
- Hypervisor and virtualization layer
- Core network infrastructure and global backbone
- Managed service software (e.g., RDS database engine patching, Lambda runtime updates)

What the Customer Manages

- Operating system patches and updates on EC2 instances
- Application code, libraries, and frameworks
- Network configuration: Security Groups, NACLs, routing
- Identity and access management: IAM users, roles, and policies
- Data encryption at rest and in transit
- Firewall configuration and port management

REAL-WORLD EXAMPLE

A company running a web application on EC2 must patch the Linux OS, configure the web server securely, and manage IAM permissions — even though AWS guarantees the physical

server underneath is secure. The 2019 Capital One breach was partly due to misconfigured firewall rules — an area squarely in the customer's responsibility zone.

Layer	AWS Responsibility	Customer Responsibility
Physical	100% AWS	None
Hypervisor/Networking	100% AWS	None
OS (EC2)	Provides AMI	Patch & harden
OS (RDS, Lambda)	Fully managed	None
Application	Provides platform	Secure the code
Data	Storage durability	Encryption & classification
IAM	IAM service itself	Policies & access control

1.2 AWS Global Infrastructure & Security

AWS operates in Regions and Availability Zones (AZs) spread across the globe. Understanding the infrastructure helps make better security and resilience decisions.

Regions

An AWS Region is a geographic area (e.g., us-east-1 for N. Virginia, ap-south-1 for Mumbai) containing multiple, isolated data centers. Data stored in a Region does not leave that Region unless the customer explicitly moves it. This is critical for data residency and compliance requirements such as GDPR.

Availability Zones

Each Region has multiple Availability Zones (typically 3 or more). AZs are physically separate facilities with independent power, cooling, and networking — yet connected with ultra-low-latency private links. Distributing workloads across AZs protects against localized failures.

AWS Edge Locations

AWS has over 400 edge locations (Points of Presence) worldwide used by services like CloudFront (CDN) and Route 53 (DNS). These locations bring content closer to users, but also serve as a security perimeter for services like AWS Shield and AWS WAF.

SECURITY INSIGHT

Choosing the right AWS Region is a foundational security decision. Consider: data sovereignty laws, proximity to users (latency), available services, and compliance certifications (some Regions are FedRAMP-authorized for US government use).

1.3 Core Security Principles in AWS

AWS security is built upon six core principles that guide every architectural decision. These principles, derived from decades of enterprise security experience, form the backbone of the AWS Well-Architected Framework's Security Pillar.

1. **Implement a Strong Identity Foundation** — Apply the principle of least privilege. Every human and machine identity should have only the permissions it needs to do its job, nothing more.
2. **Enable Traceability** — Log everything. Use CloudTrail, VPC Flow Logs, and CloudWatch to maintain a complete audit trail of who did what, when, and from where.
3. **Apply Security at All Layers** — Don't rely on a single perimeter. Secure the network edge, every subnet, every instance, every application, and every data store independently.
4. **Automate Security Best Practices** — Use Infrastructure as Code (IaC), AWS Config Rules, and automated remediation to enforce security standards at scale without human error.
5. **Protect Data in Transit and at Rest** — Classify your data sensitivity levels and apply appropriate encryption mechanisms everywhere data flows or resides.
6. **Prepare for Security Events** — Have an incident response plan, conduct game days (simulated drills), and use AWS tools to detect, contain, and recover from incidents quickly.

1.4 The AWS Well-Architected Security Pillar

The AWS Well-Architected Framework is a set of best practices developed by AWS Solutions Architects based on lessons learned from thousands of cloud deployments. The Security Pillar is one of six pillars in this framework.

The Security Pillar is organized into seven focus areas:

Focus Area	Description	Key AWS Services
Security Foundations	Governance, risk, compliance baseline	Organizations, Control Tower
Identity & Access Mgmt	Authentication and authorization controls	IAM, SSO, Cognito
Detection	Identifying threats and anomalies	GuardDuty, Security Hub, Macie

Focus Area	Description	Key AWS Services
Infrastructure Protection	Network and compute layer defense	VPC, WAF, Shield, Firewall
Data Protection	Encryption and data lifecycle management	KMS, ACM, Macie, S3
Incident Response	Preparation and response to security events	CloudTrail, Detective, Lambda
Application Security	Securing code and APIs	WAF, CodeGuru, Inspector

WELL-ARCHITECTED REVIEW

AWS offers free Well-Architected Reviews through AWS Partners or directly with AWS. Running a security review against this framework can identify gaps before attackers do. The AWS Well-Architected Tool in the console allows self-assessments across all pillars.

02

Identity and Access Management (IAM)

Identity and Access Management (IAM) is the gatekeeper of your AWS environment. Every action taken in AWS — whether by a human, an application, or a service — must be authenticated and authorized through IAM. Misconfigured IAM is the leading cause of cloud security incidents.

2.1 IAM Fundamentals: Users, Groups, Roles, and Policies

IAM Users

An IAM User is a permanent identity created for a person or application that needs long-term access to AWS. Each user has a unique name and can have credentials (password for console access, access keys for API/CLI access).

BEST PRACTICE

Avoid creating long-lived IAM access keys. Prefer IAM Roles for applications and use temporary credentials wherever possible. If access keys are necessary, rotate them every 90 days.

IAM Groups

IAM Groups are collections of users. Assigning permissions to groups rather than individual users makes management scalable. For example, create a 'Developers' group with appropriate permissions and add/remove users as staff join or leave.

IAM Roles

IAM Roles are temporary identities assumed by services, applications, or users. Unlike users, roles do not have permanent credentials. When a role is assumed, AWS issues temporary security tokens that expire. This is the preferred method for granting access to EC2 instances, Lambda functions, and cross-account access.

Identity Type	Use Case	Credential Type
IAM User	Human administrators with console access	Password + Access Keys
IAM Role (EC2)	Application on EC2 calling AWS APIs	Temporary tokens (auto-rotated)
IAM Role (Cross-Account)	Account A accessing resources in Account B	Temporary tokens via STS
IAM Role (Lambda)	Serverless function accessing DynamoDB or S3	Temporary tokens (auto-rotated)

Identity Type	Use Case	Credential Type
Service-Linked Role	AWS service acting on your behalf (e.g., ECS)	Managed by AWS

IAM Policies

Policies are JSON documents that define what actions are allowed or denied on which resources under what conditions. There are several types:

- Identity-based policies: Attached to users, groups, or roles
- Resource-based policies: Attached to resources like S3 buckets or KMS keys (allow cross-account access)
- Permission boundaries: Set the maximum permissions an identity can have (used for delegation)
- Service Control Policies (SCPs): Organizational-level guardrails applied through AWS Organizations
- Session policies: Temporary restrictions passed at role assumption time

Example IAM Policy (Least-Privilege S3 Read):

IAM POLICY EXAMPLE

```
{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["s3:GetObject", "s3:ListBucket"], "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"], "Condition": { "StringEquals": { "aws:RequestedRegion": "us-east-1" } } } ] }
```

2.2 IAM Best Practices

- Lock away the root account: The root account has unlimited power. Enable MFA on root immediately, do not create access keys for root, and store root credentials in a secure vault.
- Apply least privilege: Grant only the permissions required for a specific task. Start with zero permissions and add only what is needed.
- Use IAM Roles for EC2 and Lambda: Never embed access keys in code or configuration files. Attach IAM roles instead.
- Enable MFA for all human users: Especially for console access and privileged operations.
- Regularly review and rotate credentials: Use the IAM Credential Report to audit users and their last activity.
- Use IAM Access Analyzer: Automatically detects policies that grant access to external entities (potential data exfiltration paths).
- Tag IAM resources: Apply tags for cost allocation, security auditing, and automated governance.
- Separate duties: Use different accounts or roles for development, staging, and production environments.

SECURITY ALERT

The IAM Credential Report (downloadable from the IAM console) shows when each user last used their credentials. Any account inactive for 90+ days should be disabled. Unused access keys are a prime target for attackers.

2.3 AWS IAM Identity Center (SSO)

AWS IAM Identity Center (formerly AWS Single Sign-On) allows users to sign in once and access multiple AWS accounts and applications with a single set of credentials. It integrates with external identity providers (IdPs) like Okta, Azure Active Directory, and Google Workspace via SAML 2.0.

Key Benefits

- Eliminates the need to manage separate IAM users across dozens of AWS accounts
- Centralized access management for multi-account AWS Organizations
- Automatic provisioning and de-provisioning of user access
- Short-lived credentials issued per session — no long-lived access keys
- Audit trail in CloudTrail for all SSO sign-in events

2.4 Multi-Factor Authentication (MFA)

MFA adds a second layer of verification beyond username and password. Even if credentials are compromised, an attacker cannot access the account without the second factor.

MFA Type	Description	Best For
Virtual MFA (TOTP)	Google Authenticator, Authy — time-based codes	Developer and admin accounts
Hardware MFA (FIDO2)	YubiKey or similar hardware token	Privileged/root accounts
SMS MFA	One-time code sent via text message	Not recommended (SIM swapping risk)
WebAuthn	Biometrics or device PIN via browser	Passwordless authentication

Enforce MFA using IAM Condition Keys:

MFA ENFORCEMENT POLICY

"Condition": { "BoolIfExists": { "aws:MultiFactorAuthPresent": "true" } } — Add this condition to sensitive policies to deny access unless MFA has been used in the current session.

2.5 Service Control Policies and Permission Boundaries

Service Control Policies (SCPs) are one of the most powerful enterprise security controls in AWS. They operate at the AWS Organizations level and set the maximum permissions available to any account within an Organizational Unit (OU), regardless of what IAM policies say.

Example SCP Use Cases

- Deny all actions outside of approved AWS Regions (e.g., restrict to us-east-1 and eu-west-1 only)
- Prevent disabling of CloudTrail logging
- Block creation of public S3 buckets
- Deny IAM root account actions except specific emergency scenarios
- Restrict which EC2 instance types can be launched to prevent runaway costs

IMPORTANT NOTE

SCPs do not grant permissions — they only restrict them. A Deny in an SCP cannot be overridden by any IAM policy in any account under that OU. This makes SCPs excellent for non-negotiable security guardrails.

03

Network Security

Network security in AWS is built on the concept of defense in depth — multiple layers of controls that an attacker must defeat individually. AWS provides rich networking tools that allow precise control over who can communicate with what, in which direction, and using which protocols.

3.1 Amazon VPC Architecture & Security

Amazon Virtual Private Cloud (VPC) is a logically isolated section of the AWS cloud where you launch resources in a virtual network that you define. Every security-conscious AWS architecture starts with careful VPC design.

VPC Security Components

- Subnets: Public (internet-routable) and Private (no direct internet access). Place sensitive resources like databases exclusively in private subnets.
- Internet Gateway (IGW): Enables internet access for public subnets. Private subnets should NEVER have a route to an IGW.
- NAT Gateway: Allows private subnet resources to initiate outbound internet connections (e.g., to download patches) without accepting inbound connections.
- VPC Flow Logs: Records all IP traffic in/out of network interfaces. Essential for security monitoring and forensics.
- VPC Endpoints: Private connections from your VPC to AWS services (S3, DynamoDB) without traversing the internet.

Subnet Type	Internet Access	Typical Use
Public	Inbound + Outbound via IGW	Load balancers, bastion hosts, NAT Gateways
Private	Outbound only via NAT Gateway	Application servers, containers, EC2 workloads
Isolated/DB	None	RDS databases, ElastiCache, internal data stores

3.2 Security Groups and Network ACLs

These are two distinct, complementary network filtering mechanisms in AWS. Understanding the difference is fundamental to building secure network architectures.

Feature	Security Groups	Network ACLs
Scope	Applied to instances/ENIs	Applied to subnets
State	Stateful (return traffic auto-allowed)	Stateless (must allow both directions)
Rules	Allow rules only	Allow AND Deny rules
Evaluation	All rules evaluated together	Rules evaluated in number order
Default	Deny all inbound, allow all outbound	Allow all traffic
Best Use	Fine-grained instance-level control	Broad subnet-level controls/explicit denies

Security Group Best Practices

- Never use 0.0.0.0/0 (all IPs) as a source for SSH (port 22) or RDP (port 3389)
- Reference other Security Group IDs as sources, not IP ranges, for internal traffic
- Use separate Security Groups for each tier (web, application, database)
- Audit and remove unused Security Groups regularly
- Apply the principle of least privilege: open only the ports and protocols actually needed

3.3 AWS WAF, Shield, and DDoS Protection

AWS WAF (Web Application Firewall)

AWS WAF inspects HTTP/HTTPS requests at Layer 7 (application layer) and blocks malicious traffic based on rules. It integrates with CloudFront, Application Load Balancers, API Gateway, and AppSync.

What AWS WAF Can Block

- SQL Injection (SQLi) attacks — malicious database commands embedded in web requests
- Cross-Site Scripting (XSS) — injecting malicious scripts into web pages
- Geographic restrictions — block or allow traffic from specific countries
- Rate limiting — automatically block IPs sending too many requests per second
- Bot traffic — using AWS Managed Rules for Bot Control
- Known malicious IPs — using AWS IP Reputation lists

AWS Shield

- AWS Shield Standard: Free, automatically protects all AWS resources from common volumetric DDoS attacks (Layer 3 and Layer 4).

- AWS Shield Advanced: Paid service (\$3,000/month + data transfer fees) providing 24/7 access to the DDoS Response Team (DRT), near-real-time attack visibility, financial protection against DDoS-related cost spikes, and WAF integration.

RECOMMENDATION

For any internet-facing application handling sensitive data or with strict uptime requirements, AWS Shield Advanced is strongly recommended. The DRT can proactively engage during attacks and the financial protection clause can save significant costs during large-scale DDoS events.

3.4 AWS Network Firewall & Traffic Inspection

AWS Network Firewall is a managed service providing deep packet inspection, stateful firewall rules, and intrusion detection/prevention (IDS/IPS) for VPC traffic. Unlike Security Groups (which operate at the instance level), Network Firewall inspects all traffic at the VPC perimeter.

Key Capabilities

- Stateful and stateless rule groups for fine-grained traffic control
- Suricata-compatible IDS/IPS rules for threat detection
- Domain name filtering — block outbound connections to malicious domains (DNS exfiltration prevention)
- TLS inspection (decrypt, inspect, re-encrypt) for encrypted traffic analysis
- Centralized logging to S3, CloudWatch, or Kinesis Firehose

3.5 VPN, Direct Connect, and Private Connectivity

AWS Site-to-Site VPN

Creates encrypted IPsec tunnels between your on-premises network and your VPC. Traffic travels over the public internet but is encrypted end-to-end. Ideal for organizations starting their cloud journey or needing a backup connection.

AWS Direct Connect

A dedicated private network connection from your data center to AWS — traffic never touches the public internet. Provides more consistent network performance, lower latency, and often lower data transfer costs for high-volume workloads.

AWS PrivateLink

Enables private connectivity between VPCs and AWS services without requiring internet gateways, NAT devices, or VPN connections. Traffic stays within the AWS network, never crossing the public internet — critical for compliance-sensitive data flows.

ARCHITECTURAL BEST PRACTICE

For any production workload handling sensitive data, design your architecture so that traffic between application servers and databases NEVER leaves your VPC. Use VPC Endpoints for S3 and DynamoDB, PrivateLink for other services, and restrict outbound internet access to only what is necessary via NAT Gateways.

04**Data Protection**

Data is the most valuable asset organizations store in the cloud. Protecting data means ensuring its confidentiality (only authorized parties can read it), integrity (it hasn't been tampered with), and availability (it's accessible when needed). AWS provides a comprehensive suite of tools to achieve all three.

4.1 Encryption at Rest

Encryption at rest means data is encrypted when it is stored — on disks, in databases, in object storage, or in backups. If an attacker gains physical access to the storage medium, they cannot read the data without the encryption key.

AWS Services with Native Encryption at Rest

- Amazon S3: Server-Side Encryption (SSE) with S3-managed keys (SSE-S3), KMS keys (SSE-KMS), or customer-provided keys (SSE-C)
- Amazon EBS: AES-256 encryption for EC2 block storage volumes; encrypted snapshots automatically propagate encryption
- Amazon RDS: One-click encryption for database instances using KMS; automatic encryption of backups and read replicas
- Amazon DynamoDB: Encryption at rest enabled by default using KMS
- Amazon EFS: KMS-based encryption for elastic file system
- AWS Backup: All backups encrypted using the service's KMS key

ENFORCEMENT TIP

Use AWS Config Rule 'encrypted-volumes' to automatically flag any unencrypted EBS volumes. Create an SCP to deny creation of unencrypted RDS instances. Enable S3 Block Public Access at the account level and require SSE-KMS on all buckets via bucket policies.

4.2 Encryption in Transit

Data in transit — moving between a client and server, between microservices, or between AWS services — must be encrypted to prevent interception (man-in-the-middle attacks, packet sniffing).

TLS/SSL Certificates with AWS Certificate Manager (ACM)

ACM provides free public TLS certificates for use with AWS services. ACM handles automatic renewal, eliminating a common source of outages (expired certificates). For internal/private communications, ACM Private CA issues private certificates.

In-Transit Encryption Across AWS Services

- Load Balancers (ALB/NLB): Terminate TLS at the load balancer and optionally re-encrypt to backend instances
- API Gateway: Enforces HTTPS; can enforce minimum TLS version (TLS 1.2 or 1.3)
- CloudFront: Supports TLS 1.3 for edge connections; can enforce HTTPS-only viewer policies
- RDS: Enforce SSL connections via parameter group settings
- Elasticsearch/OpenSearch: Enable node-to-node encryption and require HTTPS

4.3 AWS Key Management Service (KMS)

AWS KMS is the central key management service for AWS encryption. It creates and manages cryptographic keys and controls their use across AWS services and your applications. KMS is FIPS 140-2 validated and uses Hardware Security Modules (HSMs) to protect keys.

Types of KMS Keys

Key Type	Who Creates/Manages	Use Case
AWS Managed Keys	AWS creates and rotates automatically	Default encryption for AWS services
Customer Managed Keys (CMK)	Customer creates; AWS stores securely	Fine-grained control, auditing, custom rotation
AWS CloudHSM Keys	Customer-owned dedicated HSM	Regulatory requirements for key ownership
Data Keys	Generated by KMS on request	Envelope encryption for large data

KMS Key Policies

Every KMS key has a resource-based policy (Key Policy) that defines who can use and manage the key. This is separate from IAM policies and provides an additional access control layer. Key policies are the definitive authority — even if an IAM policy allows KMS actions, the key policy must also permit them.

SECURITY BEST PRACTICE

Enable automatic key rotation for all Customer Managed KMS Keys (set to every 365 days). Monitor KMS key usage with CloudTrail — every encryption and decryption operation is logged. Set up CloudWatch alarms for unusual KMS API call volumes, which may indicate a data exfiltration attempt.

4.4 AWS Secrets Manager & Parameter Store

Hardcoded credentials in application code or configuration files are one of the most common and dangerous security vulnerabilities. AWS provides two services to securely store and retrieve secrets:

AWS Secrets Manager

- Designed specifically for credentials like database passwords, API keys, OAuth tokens
- Automatic rotation of secrets (built-in rotation for RDS, Redshift, DocumentDB; Lambda-based rotation for custom secrets)
- Versioning: stores previous secret versions during rotation for seamless application transitions
- Fine-grained access control via IAM and resource-based policies
- Audit trail via CloudTrail
- Cost: \$0.40 per secret per month + \$0.05 per 10,000 API calls

AWS Systems Manager Parameter Store

- Stores configuration data and secrets as name-value pairs
- Standard tier is free; Advanced tier supports larger values and policies
- Integration with KMS for encrypted SecureString parameters
- Hierarchical naming convention (e.g., /myapp/prod/db-password) for organized management

GUIDANCE

Use Secrets Manager when you need automatic credential rotation (especially for databases). Use Parameter Store for general configuration data and less sensitive application settings. Never store secrets in environment variables, source code, or EC2 user data scripts.

4.5 Data Classification and S3 Security

Amazon S3 is one of the most widely used services in AWS and also one of the most common sources of data breaches — typically due to misconfiguration. A rigorous approach to S3 security is essential.

Data Classification Framework

- Public: Data intended for public consumption (marketing materials, public documentation)
- Internal: Non-sensitive internal data (meeting notes, internal wikis)
- Confidential: Business-sensitive data (financial records, employee data)
- Restricted: Highly sensitive regulated data (health records, payment card data, PII)

S3 Security Controls Checklist

- Enable S3 Block Public Access at the account level and for all buckets
- Enable S3 Server Access Logging for all buckets containing sensitive data
- Enable S3 Versioning to protect against accidental deletion and ransomware
- Enable S3 Object Lock (WORM) for compliance data that must not be modified or deleted
- Use S3 Bucket Policies to enforce encryption (deny PutObject without server-side encryption)
- Enable Amazon Macie to automatically discover and classify sensitive data in S3
- Use S3 Access Points to manage access for different use cases without complex bucket policies
- Enable MFA Delete for S3 buckets containing critical data

05

Security Best Practices for Core AWS Services

Each AWS service has unique security considerations. This section covers the most widely-used services and the specific security controls that should be applied when deploying them in production environments.

5.1 EC2 Security Hardening

Amazon EC2 (Elastic Compute Cloud) instances are virtual machines running in the cloud. Because they run general-purpose operating systems, they require active security management.

OS-Level Hardening

- Use AWS-provided, regularly updated Amazon Machine Images (AMIs) or create hardened golden AMIs using EC2 Image Builder
- Apply OS patches regularly; use AWS Systems Manager Patch Manager for automated, scheduled patching
- Disable unnecessary services, ports, and user accounts
- Enable OS-level audit logging (auditd on Linux, Windows Event Logging on Windows)
- Install and configure endpoint protection (CrowdStrike, Trend Micro, or Amazon Inspector agent)

Access Control for EC2

- Use EC2 Instance Connect or AWS Systems Manager Session Manager instead of SSH (no open port 22 required)
- If SSH is required, use SSH key pairs and restrict security group rules to specific IP ranges
- Use IAM Instance Profiles (roles) rather than access keys for AWS API access from the instance
- Implement a Bastion Host or AWS Systems Manager Jump Server for accessing private instances

Instance Metadata Service (IMDS)

The EC2 Instance Metadata Service provides information about the running instance, including temporary IAM credentials. IMDSv1 is vulnerable to Server-Side Request Forgery (SSRF) attacks. Always enforce IMDSv2, which requires a session token:

SECURITY REQUIREMENT

Enforce IMDSv2 on all EC2 instances. In the AWS Console: EC2 > Instance > Metadata Options > set 'IMDSv2 = Required'. Use an SCP or AWS Config Rule to enforce this across all accounts. This single change would have mitigated the Capital One breach vector.

5.2 RDS and Database Security

Amazon RDS (Relational Database Service) manages common relational databases (MySQL, PostgreSQL, Oracle, SQL Server, MariaDB). Because databases hold the most sensitive data, their security requires special attention.

- Deploy RDS in private subnets — never in public subnets with public accessibility enabled
- Enable encryption at rest using KMS from the time of creation (cannot be added to an existing unencrypted instance without migration)
- Enforce SSL/TLS connections; set `rds.force_ssl` parameter to 1 for PostgreSQL
- Use IAM database authentication to avoid static username/password credentials
- Enable Enhanced Monitoring and Performance Insights for anomaly detection
- Enable automatic backups and test restoration procedures regularly
- Use database activity streams for near-real-time monitoring of database activity (available for Aurora)
- Restrict Security Group rules to only the application server Security Group — not the entire VPC CIDR
- Enable multi-AZ deployment for high availability and protection against AZ failures

5.3 Lambda and Serverless Security

AWS Lambda runs code in response to events without requiring server management. The security model is different from EC2 — you're responsible for the code itself, its dependencies, and the IAM permissions granted to the function.

Lambda Security Fundamentals

- Principle of Least Privilege: Each Lambda function should have its own execution role with minimal permissions — only what that specific function needs
- Environment Variables: Store non-sensitive config in environment variables; use Secrets Manager or Parameter Store (SecureString) for sensitive values
- Code Security: Scan dependencies for vulnerabilities using Amazon CodeGuru Reviewer or third-party tools like Snyk
- VPC Deployment: Place Lambda in a VPC for access to private resources; be aware that VPC Lambda needs a NAT Gateway for internet access
- Function URL Security: If using Lambda function URLs, enforce `AWS_IAM` auth type rather than `NONE`

- Layer Security: Audit Lambda Layers for vulnerable or malicious packages

SERVERLESS INSIGHT

In a serverless environment, the attack surface shifts from the OS to the application code and its dependencies. Software Composition Analysis (SCA) — scanning all third-party libraries for known CVEs — is critical for Lambda security. A single vulnerable NPM package can expose your entire function.

5.4 Container Security (ECS/EKS)

AWS offers two managed container orchestration services: ECS (Elastic Container Service) and EKS (Elastic Kubernetes Service). Container security requires attention at the image, runtime, and orchestration layers.

Container Image Security

- Use Amazon ECR (Elastic Container Registry) to store container images; enable enhanced scanning with Inspector to detect OS and programming language vulnerabilities
- Use minimal base images (Alpine, Distroless) to reduce the attack surface
- Never run containers as root; specify a non-root user in your Dockerfile
- Sign container images using AWS Signer or Notary and enforce image signing policies
- Regularly rebuild images to incorporate security patches

Runtime Security for EKS

- Use AWS Fargate where possible (AWS manages the underlying EC2 instance, reducing your attack surface)
- Apply Kubernetes RBAC (Role-Based Access Control) — give each service account minimal permissions
- Enable EKS Control Plane Logging (API, audit, authenticator, controller manager, scheduler logs)
- Use Amazon GuardDuty EKS Protection for threat detection at the Kubernetes API layer
- Apply Pod Security Standards (restricted profile) to prevent privilege escalation
- Use OPA (Open Policy Agent) or Kyverno for policy-as-code enforcement

5.5 S3 Bucket Security Best Practices

While covered in the Data Protection section, S3 security deserves expanded attention because S3 misconfiguration is the #1 cause of cloud data breaches. The following is a comprehensive checklist:

Control	Implementation	Priority
Block Public Access	Enable at account level + bucket level	CRITICAL
Bucket Encryption	Set default encryption to SSE-KMS	HIGH
Bucket Policy: HTTPS Only	Deny HTTP (non-HTTPS) requests	HIGH
Bucket Policy: Enforce Encryption	Deny PutObject without SSE-KMS header	HIGH
Access Logging	Enable on all sensitive buckets	HIGH
Versioning	Enable on all production buckets	MEDIUM
Object Lock (WORM)	Enable for compliance/archive data	MEDIUM
MFA Delete	Enable for critical data buckets	MEDIUM
Intelligent Tiering	Manage data lifecycle and reduce costs	LOW
Replication (Cross-Region)	Disaster recovery for critical data	MEDIUM

06

Compliance and Regulatory Standards

Operating in a regulated industry requires demonstrating that your cloud environment meets specific security standards. AWS maintains a large portfolio of compliance certifications and provides services and documentation to help customers meet their own compliance obligations.

6.1 AWS Compliance Programs Overview

AWS has achieved an extensive list of global compliance certifications that validate the security of its infrastructure. Customers can leverage these certifications to reduce their own compliance burden — this is called 'inheriting' AWS compliance.

Standard/Framework	Industry	Key Requirement Areas
PCI DSS Level 1	Payment Cards	Cardholder data protection, access control, monitoring
HIPAA	Healthcare (US)	PHI protection, access controls, audit logs, encryption
SOC 1, 2, 3	General (Financial/SaaS)	Security, availability, processing integrity
ISO 27001	Global/General	Information security management system
FedRAMP	US Federal Government	NIST 800-53 controls for government data
GDPR	EU Personal Data	Data sovereignty, consent, right to erasure
HITRUST	Healthcare/Financial	Healthcare data security framework
CSA STAR	Cloud	Cloud-specific security best practices

AWS ARTIFACT

AWS Artifact is a free self-service portal in the AWS Console that provides on-demand access to AWS compliance reports (SOC, PCI, ISO reports) and compliance agreements (BAA for HIPAA, NDA). Download these directly from Artifact rather than waiting for AWS to send them manually.

6.2 PCI DSS on AWS

PCI DSS (Payment Card Industry Data Security Standard) is a mandatory standard for any organization that handles cardholder data (credit/debit card numbers, CVVs, PINs). AWS is a PCI DSS Level 1 Service Provider, the highest level of PCI certification.

Key PCI DSS Requirements Mapped to AWS

PCI Requirement	AWS Implementation
Req 1: Firewall	VPC Security Groups, Network ACLs, AWS Network Firewall
Req 2: No defaults	Custom AMIs, hardened configs, Secrets Manager (no default passwords)
Req 3: Protect stored data	KMS encryption, S3 SSE-KMS, RDS encryption
Req 4: Encrypt in transit	ACM/TLS for all endpoints, enforce HTTPS
Req 5: Anti-malware	Amazon Inspector, EC2 AV software, GuardDuty
Req 6: Secure systems	AWS Systems Manager Patch Manager, CodePipeline security scans
Req 7: Restrict access	IAM least privilege, SCPs, resource-based policies
Req 8: Identify and authenticate	IAM MFA, IAM Identity Center, MFA enforcement
Req 9: Physical access	AWS manages physical access — inherited
Req 10: Monitor access	CloudTrail, CloudWatch, Security Hub, VPC Flow Logs
Req 11: Test security	Amazon Inspector, AWS Config, Penetration Testing
Req 12: Security policy	AWS Config Rules, Security Hub standards, documentation

6.3 HIPAA on AWS

HIPAA (Health Insurance Portability and Accountability Act) protects the privacy and security of Protected Health Information (PHI) in the United States. AWS offers HIPAA-eligible services, but customers are responsible for configuring them correctly.

AWS HIPAA Requirements

- Sign a Business Associate Agreement (BAA) with AWS — downloadable from AWS Artifact
- Only process PHI on HIPAA-eligible AWS services (over 100 services qualify, including EC2, RDS, S3, Lambda, and many more)
- Enable encryption at rest (KMS) and in transit (TLS) for all PHI
- Implement comprehensive access logging with CloudTrail
- Enable MFA for all users with access to PHI
- Maintain backup and disaster recovery plans

- Conduct annual security risk assessments

IMPORTANT

HIPAA does not certify cloud providers — there is no 'HIPAA certification' for AWS. Instead, AWS provides the tools and BAA. It is the healthcare organization's responsibility to implement the controls correctly. AWS Audit Manager includes a HIPAA framework template to help assess compliance.

6.4 SOC 2 on AWS

SOC 2 (System and Organization Controls 2) is an auditing framework for service providers that store customer data in the cloud. It evaluates controls related to Security (mandatory), Availability, Processing Integrity, Confidentiality, and Privacy Trust Service Criteria.

AWS provides its own SOC 2 Type II report (available via Artifact), which covers the security of AWS infrastructure. For SaaS companies building on AWS, this means the infrastructure layer compliance is inherited. The application and data layer controls must still be implemented and independently audited.

AWS Services Useful for SOC 2 Compliance

- CloudTrail: Provides the audit log evidence auditors require
- AWS Config: Demonstrates configuration change management and compliance monitoring
- AWS Security Hub: Aggregates findings to demonstrate continuous security monitoring
- IAM Credential Report: Evidence of access control reviews
- AWS Audit Manager: Automates evidence collection for SOC 2 audits

6.5 GDPR and Data Residency on AWS

GDPR (General Data Protection Regulation) is the EU's comprehensive data privacy law. It applies to any organization processing personal data of EU residents, regardless of where the organization is located.

AWS Tools for GDPR Compliance

- AWS Regions: Keep EU personal data in EU Regions (eu-west-1, eu-central-1, etc.) — data doesn't leave the Region unless you explicitly configure it to
- Amazon Macie: Automatically discovers and classifies personal data (PII) in S3
- AWS Data Lifecycle Manager: Automates deletion of data at defined lifecycle points (right to erasure)

- VPC Flow Logs + CloudTrail: Provides audit trails for data access (required for breach notification)
- AWS Config: Tracks configuration changes for accountability
- AWS Audit Manager: GDPR framework template for evidence collection

07**Security Monitoring and Auditing**

Security monitoring is the practice of continuously observing your AWS environment to detect threats, anomalies, and compliance violations. AWS provides a rich set of native monitoring tools that, when used together, form a comprehensive security observability platform.

7.1 AWS CloudTrail

AWS CloudTrail records every API call made in your AWS account — who did what, when, from which IP address, and using which tool (console, CLI, SDK, or service). It is the foundation of AWS security auditing and forensics.

CloudTrail Best Practices

- Enable CloudTrail in ALL Regions (use a multi-region trail to capture all activity)
- Enable CloudTrail Log File Validation to detect tampering with log files
- Store CloudTrail logs in a dedicated, centralized S3 bucket in a separate security account
- Protect the log bucket: deny delete operations, enable versioning, enable MFA Delete
- Set up CloudWatch Alarms for critical CloudTrail events (root account login, security group changes, IAM policy changes)
- Enable CloudTrail Insights to automatically detect unusual API activity patterns
- Retain logs for at least 1 year (HIPAA, PCI) or longer based on your compliance requirements

CRITICAL EVENTS TO MONITOR IN CLOUDTRAIL

Root account login without MFA, IAM policy changes, Security Group and NACL changes, S3 bucket policy changes, KMS key deletion, CloudTrail itself being disabled or modified, creation of new IAM users or roles.

7.2 Amazon GuardDuty

Amazon GuardDuty is a managed threat detection service that uses machine learning, anomaly detection, and threat intelligence feeds to identify malicious activity and unauthorized behavior in your AWS environment. Unlike rule-based tools, GuardDuty learns normal patterns for your account and flags deviations.

What GuardDuty Analyzes

- AWS CloudTrail Event Logs — detects unusual API calls, IAM credential misuse

- VPC Flow Logs — identifies unusual network traffic patterns, cryptocurrency mining
- DNS Logs — detects communication with known malicious domains
- EKS Audit Logs — detects suspicious activity in Kubernetes clusters (with EKS Protection enabled)
- S3 Data Events — detects unusual data access patterns (with S3 Protection enabled)
- RDS Login Events — detects brute force and anomalous login attempts (with RDS Protection)

Key GuardDuty Finding Categories

Category	Example Finding	Severity
Backdoor	EC2 instance polling known C&C domain	HIGH
Credential Access	IAM credentials used from Tor exit node	HIGH
Crypto Mining	EC2 Bitcoin mining traffic detected	HIGH
Data Exfiltration	S3 bucket accessed from unusual geography	HIGH
Privilege Escalation	EC2 instance attempting to add policies to IAM roles	MEDIUM
Recon	Port scanning originating from EC2 instance	LOW

7.3 AWS Security Hub

AWS Security Hub is the centralized command center for AWS security findings. It aggregates, normalizes, and prioritizes security findings from GuardDuty, Inspector, Macie, Firewall Manager, and third-party tools — all in one place.

Security Hub Standards

- AWS Foundational Security Best Practices (FSBP): Over 250 automated security checks covering 40+ AWS services
- CIS AWS Foundations Benchmark: The Center for Internet Security's prescriptive AWS hardening guidelines
- PCI DSS: Automated checks aligned to PCI requirements
- NIST SP 800-53: Controls mapped to NIST cybersecurity framework

MULTI-ACCOUNT STRATEGY

In a multi-account AWS Organization, designate one account as the Security Hub Administrator. All member accounts' findings flow into the administrator account, giving the security team a single pane of glass across the entire organization — without needing to log into each account individually.

7.4 Amazon Macie

Amazon Macie is a fully managed data security service that uses machine learning to automatically discover, classify, and protect sensitive data in Amazon S3. It identifies data such as personally identifiable information (PII), protected health information (PHI), financial data, and credentials.

What Macie Detects

- Names, addresses, phone numbers, email addresses, national ID numbers (PII)
- Credit card numbers, bank account numbers (financial data)
- Medical records, prescriptions (PHI)
- API keys, AWS secret access keys, SSH private keys stored in S3
- Custom data patterns via custom data identifiers (regex-based)

7.5 AWS Config and Compliance Monitoring

AWS Config records the configuration state of your AWS resources over time. You can view any resource's configuration at any point in the past and see how it changed. Config Rules allow you to define compliance rules and automatically evaluate resource configurations against them.

Essential AWS Config Rules

- `s3-bucket-public-read-prohibited`: Alerts when any S3 bucket allows public read access
- `restricted-ssh`: Alerts when Security Groups allow unrestricted SSH access (port 22)
- `root-account-mfa-enabled`: Alerts if the root account does not have MFA enabled
- `encrypted-volumes`: Alerts when EBS volumes are not encrypted
- `rds-storage-encrypted`: Alerts when RDS instances are not encrypted
- `cloudtrail-enabled`: Alerts when CloudTrail is not enabled in a Region
- `iam-password-policy`: Checks that the IAM account password policy meets requirements

AWS Config also provides Conformance Packs — collections of Config Rules and remediation actions packaged as a single unit. AWS provides managed conformance packs for CIS Benchmarks, PCI DSS, HIPAA, and more.

08

Incident Response on AWS

Security incidents are inevitable. The question is not if a security event will occur, but when and how quickly your team can detect, contain, and recover from it. AWS provides a structured approach to incident response aligned with NIST SP 800-61.

8.1 AWS Incident Response Framework

Effective incident response on AWS follows four phases:

Phase	Objective	Key AWS Tools
Preparation	Build capabilities before incidents occur	Security Hub, GuardDuty, CloudTrail, runbooks
Detection & Analysis	Identify and understand the incident	GuardDuty, Macie, CloudTrail Insights, Detective
Containment & Eradication	Stop the bleeding and remove the threat	IAM revoke, Security Groups, VPC isolation, EC2 snapshots
Recovery & Post-Incident	Restore operations and learn lessons	AWS Backup, Game Days, PIR documentation

Preparation Checklist

- Create and test an Incident Response Plan (IRP) specific to AWS environments
- Pre-create an isolated 'clean room' VPC for forensic investigation
- Create break-glass IAM roles for emergency access with enhanced logging
- Document runbooks for common incident types (credential compromise, data exfiltration, ransomware)
- Run quarterly Game Days (tabletop exercises) to test your team's response

8.2 Forensics and Investigation Tools

Amazon Detective

Amazon Detective automatically collects log data and uses graph-based machine learning models to help security teams investigate potential security issues quickly. It visualizes resource behavior and interactions over time — connecting the dots between a GuardDuty finding and the specific API calls, network traffic, and IAM activity that preceded it.

Forensic EC2 Instance Analysis

When an EC2 instance is suspected of being compromised, the standard forensic process is:

7. Isolate the instance: Remove from Auto Scaling Group, replace Security Group with 'forensic-only' group (no inbound/outbound), detach from load balancer
8. Preserve evidence: Take an EBS snapshot before any changes, capture instance memory dump if needed
9. Investigate: Attach the snapshot to a forensic instance in the isolated VPC and analyze without affecting production
10. Eradicate: Terminate the compromised instance and launch a fresh one from a trusted golden AMI
11. Document: Record all findings, timeline, and impact for the post-incident review

8.3 Automated Remediation with Lambda & EventBridge

Manual incident response is too slow for cloud environments where resources can be created and destroyed in seconds. AWS enables Security Orchestration, Automation, and Response (SOAR) patterns using native services.

Example: Automatic Isolation of Compromised EC2 Instance

When GuardDuty detects a high-severity finding on an EC2 instance:

12. GuardDuty generates a finding
13. EventBridge rule matches the finding type and severity
14. EventBridge triggers a Lambda function
15. Lambda replaces the instance's Security Groups with an isolation group (no traffic)
16. Lambda creates an EBS snapshot for forensics
17. Lambda sends an alert to the security team via SNS/Slack
18. Lambda creates a Jira/ServiceNow ticket with all finding details

AUTOMATION BENEFIT

This automated workflow can isolate a compromised instance within 60 seconds of GuardDuty detection — compared to 15-60 minutes for a human responder. In an active breach, every second counts. Build and test these automations before you need them.

09**Real-World Case Studies**

Theory and best practices are most effectively understood when applied to real-world scenarios. The following case studies illustrate both security failures and security successes in AWS environments, providing actionable lessons for security practitioners.

9.1 Case Study: The Capital One Data Breach (2019)

Background

In July 2019, Capital One disclosed a data breach affecting approximately 100 million customers in the US and 6 million in Canada. The attacker accessed names, addresses, phone numbers, email addresses, dates of birth, self-reported income, credit scores, and Social Security numbers. It is one of the largest financial sector data breaches in history.

Attack Vector

The attacker exploited a misconfigured Web Application Firewall (WAF) using a Server-Side Request Forgery (SSRF) attack. The SSRF allowed the attacker to make requests from within the Capital One environment. Crucially, the EC2 instances were running with IMDSv1 (Instance Metadata Service version 1), which does not require authentication. The attacker was able to access the instance metadata endpoint and retrieve temporary IAM role credentials. With these credentials, they listed S3 buckets and exfiltrated over 100 million records.

Contributing Factors

- IMDSv1 was in use — IMDSv2 (which requires session tokens) would have blocked this attack vector
- Overly permissive IAM role attached to the WAF EC2 instance — it had broad S3 read access it did not need
- Insufficient monitoring — the data exfiltration went undetected for months
- No anomaly detection on S3 data access patterns

Lessons Learned

- ENFORCE IMDSv2 on all EC2 instances — this is arguably the single most impactful change in EC2 security
- Apply strict least-privilege to all IAM roles attached to EC2 instances
- Enable Amazon Macie and GuardDuty S3 Protection to detect unusual data access
- Monitor S3 data egress volume — large, unusual transfers should trigger immediate alerts
- Conduct regular threat modeling exercises to identify SSRF and similar attack paths

TAKEAWAY

The Capital One breach was preventable with well-known AWS security controls that existed at the time. IMDSv2 enforcement and IAM least privilege are now considered mandatory baseline controls in any mature AWS security program.

9.2 Case Study: Securing a Multi-Account Enterprise on AWS

Scenario

A large financial services company migrates 300+ workloads to AWS over 18 months. They need to maintain PCI DSS compliance, separate development from production, grant autonomous access to 12 engineering teams, and give the central security team visibility across all accounts.

Architecture Solution

- AWS Organizations with a multi-account hierarchy: Management Account (billing/governance only), Security Account (GuardDuty admin, Security Hub admin, CloudTrail aggregation), Shared Services Account (DNS, AD Connector, CI/CD tools), and individual Workload Accounts per team/environment
- AWS Control Tower for automated account vending with security guardrails pre-configured
- SCPs at the OU level: Root OU has deny-list of dangerous actions; Workload OUs have environment-specific restrictions
- AWS IAM Identity Center for federated SSO — engineers log in with corporate Okta credentials
- AWS Security Hub in the Security Account aggregates findings from all 300+ accounts
- GuardDuty enabled in every account, with delegation to the Security Account
- CloudTrail Organization Trail sending all logs to a tamper-protected S3 bucket in the Security Account

Outcomes

- Security team gained visibility across all accounts through a single Security Hub dashboard
- Time to provision a new secure AWS account reduced from 2 weeks to 2 hours
- PCI DSS audit evidence collection automated via AWS Audit Manager
- Mean Time to Detect (MTTD) security incidents improved from days to minutes

9.3 Case Study: Building a Zero-Trust Architecture on AWS

Background

A technology company with a hybrid cloud environment (AWS + on-premises) adopts Zero Trust principles: 'Never trust, always verify.' Traditional perimeter-based security is replaced with identity-based access controls for every request, regardless of source location.

Zero-Trust Implementation on AWS

- Identity Verification: IAM Identity Center with Okta integration; all human access requires MFA; machine identities use short-lived IAM role credentials
- Network: No direct SSH/RDP to any instance; all access via AWS Systems Manager Session Manager (identity-verified, logged sessions); VPC peering replaced with AWS PrivateLink
- Device Trust: AWS Verified Access deployed for web application access — validates device posture (MDM enrollment, patch level) before granting access
- Micro-segmentation: Each application tier has its own Security Group, VPC, and IAM role; no implicit trust between services
- Continuous Verification: All API calls verified via IAM on every request; no session-based trust; GuardDuty monitors for behavioral anomalies
- Encrypt Everything: All data encrypted at rest (KMS) and in transit (TLS 1.3); internal service-to-service communication uses mutual TLS (mTLS)

ZERO TRUST INSIGHT

Zero Trust is not a single product or technology — it is a philosophy implemented through multiple overlapping controls. AWS provides all the building blocks (IAM, VPC, PrivateLink, Verified Access, KMS) but the architecture requires deliberate design and ongoing maintenance.

10

Tools, Resources & Next Steps

Building a comprehensive AWS security program requires the right tools, ongoing education, and a structured approach to continuous improvement. This section catalogs the essential toolbox and provides a recommended learning path.

10.1 AWS Native Security Toolbox

Service	Category	Primary Purpose
IAM	Identity	Access control for all AWS resources
IAM Identity Center	Identity	Federated SSO for multi-account access
AWS Organizations	Governance	Multi-account management and SCPs
AWS Control Tower	Governance	Automated landing zone with guardrails
Amazon GuardDuty	Detection	ML-powered threat detection
AWS Security Hub	Detection	Centralized security findings aggregation
Amazon Macie	Data Security	Sensitive data discovery in S3
Amazon Inspector	Vulnerability	Automated vulnerability scanning (EC2, Lambda, ECR)
AWS Config	Compliance	Configuration recording and compliance rules
AWS CloudTrail	Audit	API activity logging
VPC Flow Logs	Network	Network traffic logging
AWS WAF	Network	Web application firewall (Layer 7)
AWS Shield	Network	DDoS protection
AWS Network Firewall	Network	Stateful/stateless VPC traffic inspection
AWS KMS	Encryption	Key management for at-rest encryption
AWS ACM	Encryption	TLS certificate management
AWS Secrets Manager	Secrets	Credentials storage and automatic rotation
AWS Detective	Forensics	Security investigation and root cause analysis
AWS Audit Manager	Compliance	Automated audit evidence collection
Amazon Cognito	Identity	User authentication for web/mobile apps

10.2 Third-Party Security Tools

While AWS native tools are powerful, many organizations complement them with specialized third-party tools for deeper coverage, consolidated management, or specific use cases:

Tool	Category	Use Case
Wiz	Cloud Security Posture	Agentless vulnerability and misconfiguration scanning
Prisma Cloud (Palo Alto)	CNAPP	Cloud-native application protection platform
CrowdStrike Falcon	Endpoint/Runtime	Endpoint protection for EC2 instances and containers
Snyk	DevSecOps	Developer-first code and dependency vulnerability scanning
Lacework	Behavioral	ML-based behavioral anomaly detection
Aqua Security	Container	Container and serverless security
HashiCorp Vault	Secrets	Multi-cloud secrets management
Splunk	SIEM	Log aggregation, correlation, and threat hunting
Datadog Security	Observability	Unified monitoring and security signals

10.3 Recommended Learning Path

Building AWS security expertise is a journey. The following path is designed for practitioners at different stages:

Beginner (0-6 months)

- Complete the AWS Cloud Practitioner certification (covers cloud fundamentals)
- Study the AWS Well-Architected Security Pillar whitepaper (free on AWS)
- Complete AWS Security Essentials course (AW-ESS)
- Practice with a personal AWS account: enable GuardDuty, CloudTrail, and Config in a free tier account

Intermediate (6-18 months)

- Obtain AWS Security Specialty certification (SCS-C02) — the premier AWS security credential
- Study the CIS AWS Foundations Benchmark and implement all recommendations
- Build a multi-account AWS organization with Control Tower in a sandbox environment
- Complete a Capture the Flag (CTF) focused on cloud security (CloudGoat, flaws.cloud, flaws2.cloud)

Advanced (18+ months)

- Pursue Cloud Security certifications: CCSP (ISC2), CCSK (CSA), or CISA
- Contribute to open-source cloud security projects (Prowler, ScoutSuite, CloudMapper)
- Build and lead a cloud security program using the NIST Cybersecurity Framework
- Implement a Security Operations Center (SOC) on AWS with automated response playbooks

ESSENTIAL FREE RESOURCES

AWS Security Documentation: docs.aws.amazon.com/security | AWS Security Blog: aws.amazon.com/blogs/security | flaws.cloud: free AWS security challenge by Scott Piper | Prowler: open-source AWS security scanner (github.com/prowler-cloud/prowler) | AWS re:Inforce sessions on YouTube | SANS Cloud Security curriculum

Document Prepared by DevOps Shack | @devopsshack | 2026

This document is intended for educational purposes. Always validate security controls against the latest AWS documentation and your organization's specific compliance requirements.